



Universidade Federal
do Rio de Janeiro

Escola Politécnica

Arquitetura de Computadores

AVX

Projeto de somador/subtrator vetorial de 32 bits
com Carry-Lookahead em VHDL
operando sobre vetores de 4, 8, 16 e 32 bits

Alunos: Luiz Cavalini, Eduardo Viana, Henrique Kezen
e Rafael Maurício

Repositório: <https://github.com/LuizCavalini/AC-AVX-Adder>

Ferramenta: Digital (Neemann) + GHDL

Rio de Janeiro, 3 de junho de 2026

Sumário

1	Introdução	1
2	Arquitetura do Sistema	1
2.1	Visão geral	1
2.2	Diagrama de Blocos	2
2.3	Princípio do controle de carry	2
3	Bloco CLA4 — Somador de 4 bits	3
3.1	Carry-Lookahead vs Ripple Carry	3
3.2	Subtração por complemento de dois	4
3.3	Implementação VHDL	4
4	Top-Level vec_adder	5
4.1	Instanciação dos 8 blocos	5
5	Simulação e Verificação	6
5.1	Testbench VHDL	6
5.2	Resultado da simulação	6
5.3	Validação no Digital	6
6	Premissas e Decisões de Projeto	7
7	Análise de Recursos	7
8	Conclusão	8
9	Referências	9

Lista de Figuras

1	Diagrama de blocos do somador vetorial: 8 instâncias CLA4 com portas AND controlando a propagação de carry	2
---	--	---

1 Introdução

Este relatório descreve o projeto de um **somador/subtrator vetorial de 32 bits** implementado em VHDL, com foco em baixa latência por meio da técnica de *Carry-Lookahead* (CLA). O circuito é inspirado no conceito de instruções vetoriais presentes nas extensões AVX dos processadores x86-64, onde um único operando de 512 bits pode ser tratado como múltiplos números menores operando em paralelo.

A motivação central é a mesma das instruções como `vpaddb` e `vpaddw` do conjunto AVX-512: um único circuito aritmético deve operar sobre diferentes tamanhos de dados de forma configurável. No projeto aqui descrito, um operando de 32 bits pode ser tratado como:

- 8 vetores de 4 bits (`vecSize_i` = "00")
- 4 vetores de 8 bits (`vecSize_i` = "01")
- 2 vetores de 16 bits (`vecSize_i` = "10")
- 1 vetor de 32 bits (`vecSize_i` = "11")

A interface completa do circuito segue a especificação do enunciado:

Tabela 1: Interface do somador vetorial

Sinal	Bits	Direção	Descrição
A_i	32	entrada	Operando A
B_i	32	entrada	Operando B
mode_i	1	entrada	0 = soma, 1 = subtração
vecSize_i	2	entrada	Tamanho do vetor (00/01/10/11)
S_o	32	saída	Resultado

2 Arquitetura do Sistema

2.1 Visão geral

O circuito é composto de dois módulos hierárquicos:

1. **cla4** — somador/subtrator CLA de 4 bits, o bloco fundamental instanciado 8 vezes.
2. **vec_adder** — módulo top-level que instancia 8 blocos **cla4** e controla o fluxo de carry entre eles conforme `vecSize_i`.

2.2 Diagrama de Blocos

A Figura 1 ilustra a arquitetura completa do somador vetorial, mostrando os 8 blocos `cla4` e as portas AND que controlam o carry entre eles.

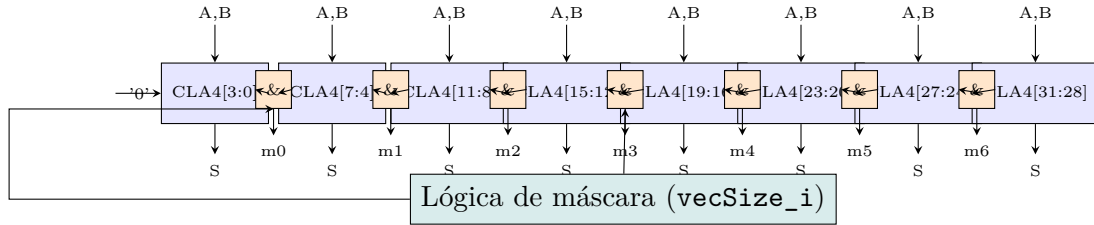


Figura 1: Diagrama de blocos do somador vetorial: 8 instâncias CLA4 com portas AND controlando a propagação de carry

2.3 Princípio do controle de carry

O segredo do somador vetorial está no **bloqueio seletivo do carry** entre blocos adjacentes. Cada bloco `cla4` produz um `cout` que normalmente alimentaria o `cin` do bloco seguinte. Uma porta AND interposta permite ou bloqueia essa propagação conforme a máscara derivada de `vecSize_i`:

$$\text{cin}_{i+1} = \text{cout}_i \text{ AND } \text{mask}_i \quad (1)$$

A Tabela 2 mostra a máscara de 7 bits para cada modo.

Tabela 2: Máscara de carry por modo de operação

<code>vecSize_i</code>	Modo	Máscara (gates 1–7)	Lanes
"00"	4 bits	0000000	8
"01"	8 bits	1010101	4
"10"	16 bits	1110111	2
"11"	32 bits	1111111	1

As equações combinacionais para cada gate são derivadas diretamente da tabela. Seja `vs(1)` e `vs(0)` os bits de `vecSize_i`:

$$\text{mask}(0) = \text{vs}(0) \text{ OR } \text{vs}(1) \quad (\text{gate entre blocos 0-1}) \quad (2)$$

$$\text{mask}(1) = \text{vs}(1) \quad (\text{gate entre blocos 1-2}) \quad (3)$$

$$\text{mask}(2) = \text{vs}(0) \text{ OR } \text{vs}(1) \quad (\text{gate entre blocos 2-3}) \quad (4)$$

$$\text{mask}(3) = \text{vs}(0) \text{ AND } \text{vs}(1) \quad (\text{gate entre blocos 3-4 — fronteira central}) \quad (5)$$

$$\text{mask}(4) = \text{vs}(0) \text{ OR } \text{vs}(1) \quad (6)$$

$$\text{mask}(5) = \text{vs}(1) \quad (7)$$

$$\text{mask}(6) = \text{vs}(0) \text{ OR } \text{vs}(1) \quad (8)$$

3 Bloco CLA4 — Somador de 4 bits

3.1 Carry-Lookahead vs Ripple Carry

Em um somador *ripple carry* de n bits, o carry se propaga sequencialmente: o bit $i + 1$ não pode ser calculado antes do carry do bit i estar disponível. Para 32 bits, isso implica latência de $32 \times t_{FA}$. O enunciado exige baixa latência, tornando essa abordagem inadequada.

O *Carry-Lookahead* elimina essa dependência calculando todos os carries em paralelo. Para cada bit i , define-se:

$$G_i = A_i \text{ AND } B_i \quad (\text{generate}) \quad (9)$$

$$P_i = A_i \text{ XOR } B_i \quad (\text{propagate}) \quad (10)$$

Os carries internos são então:

$$C_1 = G_0 \text{ OR } (P_0 \text{ AND } C_{in}) \quad (11)$$

$$C_2 = G_1 \text{ OR } (P_1 \text{ AND } G_0) \text{ OR } (P_1 \text{ AND } P_0 \text{ AND } C_{in}) \quad (12)$$

$$C_3 = G_2 \text{ OR } (P_2 \text{ AND } G_1) \text{ OR } (P_2 \text{ AND } P_1 \text{ AND } G_0) \text{ OR } (P_2 \text{ AND } P_1 \text{ AND } P_0 \text{ AND } C_{in}) \quad (13)$$

$$C_4 = G_3 \text{ OR } (P_3 \text{ AND } G_2) \text{ OR } \dots \text{ OR } (P_3 \dots P_0 \text{ AND } C_{in}) \quad (14)$$

Todas essas expressões dependem apenas de A , B e C_{in} — sem dependência em cadeia. A latência é constante: $t_{CLA} = t_{AND} + t_{OR}$, independente do número de bits.

3.2 Subtração por complemento de dois

A subtração $A - B$ é implementada como $A + (\overline{B}) + 1$.

- B é invertido bit a bit: $b_eff \leftarrow b_i \text{ XOR } (sub_i \ \& \ sub_i \ \& \ sub_i \ \& \ sub_i)$
- O $+1$ é injetado via carry de entrada: $c(0) \leftarrow cin_i \text{ OR } sub_i$

O uso de OR em vez de atribuição direta garante que, em blocos encadeados com subtração, o carry externo proveniente do bloco anterior também seja considerado corretamente.

3.3 Implementação VHDL

Listing 1: cla4 — bloco CLA de 4 bits com subtração

```

1 architecture rtl of cla4 is
2     signal b_eff : std_logic_vector(3 downto 0);
3     signal g, p : std_logic_vector(3 downto 0);
4     signal c      : std_logic_vector(4 downto 0);
5 begin
6     -- Inversão de B para subtração (complemento de dois)
7     b_eff <= b_i xor (sub_i & sub_i & sub_i & sub_i);
8
9     -- Carry de entrada: injeta +1 quando sub_i='1'
10    c(0) <= cin_i or sub_i;
11
12    -- Generate e Propagate por bit
13    g(0) <= a_i(0) and b_eff(0); p(0) <= a_i(0) xor b_eff(0);
14    g(1) <= a_i(1) and b_eff(1); p(1) <= a_i(1) xor b_eff(1);
15    g(2) <= a_i(2) and b_eff(2); p(2) <= a_i(2) xor b_eff(2);
16    g(3) <= a_i(3) and b_eff(3); p(3) <= a_i(3) xor b_eff(3);
17
18    -- Carries em paralelo (CLA)
19    c(1) <= g(0) or (p(0) and c(0));
20    c(2) <= g(1) or (p(1) and g(0)) or (p(1) and p(0) and c(0));
21    c(3) <= g(2) or (p(2) and g(1)) or (p(2) and p(1) and g(0))
22           or (p(2) and p(1) and p(0) and c(0));
23    c(4) <= g(3) or (p(3) and g(2)) or (p(3) and p(2) and g(1))
24           or (p(3) and p(2) and p(1) and g(0))
25           or (p(3) and p(2) and p(1) and p(0) and c(0));
26
27    -- Somas
28    s_o(0) <= p(0) xor c(0); s_o(1) <= p(1) xor c(1);

```

```

29   s_o(2) <= p(2) xor c(2);   s_o(3) <= p(3) xor c(3);
30
31   cout_o <= c(4);
32   g_o <= g(3) or (p(3) and g(2)) or (p(3) and p(2) and g(1))
33         or (p(3) and p(2) and p(1) and g(0));
34   p_o <= p(3) and p(2) and p(1) and p(0);
35 end architecture rtl;

```

4 Top-Level vec_adder

4.1 Instanciação dos 8 blocos

O módulo `vec_adder` instancia 8 blocos `cla4` cobrindo os bits `[31 : 0]` e interpõe as portas AND de controle de carry:

Listing 2: `vec_adder` — controle de carry e instâncias

```

1  -- Máscara derivada de vecSize_i
2  mask(0) <= vecsize_i(0) or vecsize_i(1);
3  mask(1) <= vecsize_i(1);
4  mask(2) <= vecsize_i(0) or vecsize_i(1);
5  mask(3) <= vecsize_i(0) and vecsize_i(1); -- fronteira
        central
6  mask(4) <= vecsize_i(0) or vecsize_i(1);
7  mask(5) <= vecsize_i(1);
8  mask(6) <= vecsize_i(0) or vecsize_i(1);
9
10 -- Gates de carry entre blocos
11 cin(0) <= '0';
12 cin(1) <= cout(0) and mask(0);
13 cin(2) <= cout(1) and mask(1);
14 cin(3) <= cout(2) and mask(2);
15 cin(4) <= cout(3) and mask(3);
16 cin(5) <= cout(4) and mask(4);
17 cin(6) <= cout(5) and mask(5);
18 cin(7) <= cout(6) and mask(6);
19
20 -- Instâncias (bloco 0 mostrado; blocos 1-7 seguem o mesmo
        padrão)
21 blk0: entity work.cla4 port map (
22     a_i => a_i(3 downto 0),   b_i => b_i(3 downto 0),
23     cin_i => cin(0),           sub_i => mode_i,

```



```

24   s_o => s_o(3 downto 0),    cout_o => cout(0),
25   g_o => open,                p_o => open);

```

5 Simulação e Verificação

5.1 Testbench VHDL

O testbench `vec_adder_tb.vhd` cobre os quatro modos com casos de adição, subtração e verificação de isolamento de carry. A Tabela 3 resume os casos de teste executados com GHDL.

Tabela 3: Casos de teste — `vec_adder_tb`

vecSize	mode	A	B	S esperado	Propósito
"00"	add	0x12345678	0x11111111	0x23456789	adição por nibble
"00"	add	0xFFFFFFFF	0x11111111	0x00000000	carry isolado (4b)
"01"	add	0x0A0B0C0D	0x01020304	0x0B0D0F11	adição por byte
"01"	add	0xFF0000FF	0x01000001	0x00000000	carry isolado (8b)
"10"	add	0x00010002	0x00030004	0x00040006	adição por half
"10"	add	0xFFFF0000	0x00010000	0x00000000	carry isolado (16b)
"11"	add	0xFFFF0001	0x0000FFFF	0x00000000	overflow 32b
"11"	add	0x00000001	0x00000001	0x00000002	adição simples
"11"	sub	0x00000005	0x00000003	0x00000002	subtração 32b
"11"	sub	0x00000010	0x00000010	0x00000000	subtração zero
"01"	sub	0xFF200F0A	0x01100F05	0xFE100005	subtração por byte

5.2 Resultado da simulação

Todos os 11 casos de teste passaram sem erros ao executar com GHDL:

```

$ ghdl -a src/cla4.vhd
$ ghdl -a src/vec_adder.vhd
$ ghdl -a src/vec_adder_tb.vhd
$ ghdl -e vec_adder_tb
$ ghdl -r vec_adder_tb
src/vec_adder_tb.vhd:106:5:@110ns:(report note): Todos os testes passaram!

```

5.3 Validação no Digital

O circuito foi importado no simulador Digital via componente VHDL externo com backend GHDL. Três casos foram verificados manualmente em modo interativo:

Tabela 4: Verificação interativa no Digital

Teste	A_i	B_i	Configuração	S_o
$1 + 1 = 2$	0x00000001	0x00000001	mode=0, vs="11"	0x00000002
Carry bloqueado	0xFFFFFFFF	0x11111111	mode=0, vs="00"	0x00000000
$5 - 3 = 2$	0x00000005	0x00000003	mode=1, vs="11"	0x00000002

6 Premissas e Decisões de Projeto

1. **Carry-Lookahead de 4 bits como bloco base.** A escolha de blocos de 4 bits divide exatamente os 32 bits em 8 segmentos, cobrindo todos os tamanhos vetoriais solicitados (4, 8, 16, 32). Um bloco maior (ex: 8 bits) também funcionaria mas aumentaria a complexidade das equações CLA sem ganho para este projeto.
2. **Máscara combinacional pura.** Os gates de carry são implementados como lógica puramente combinacional, sem registradores. Isso garante latência mínima e atende ao requisito de circuito orientado a desempenho.
3. **Subtração via complemento de dois.** A inversão de B e a injeção do +1 via cin 0R sub é a abordagem padrão em ALUs e permite reutilizar toda a lógica de adição sem circuito adicional.
4. **Saídas G e P do bloco.** O cla4 expõe g_o e p_o de bloco inteiro, permitindo uma extensão futura para CLA hierárquico (como o 74S182), caso se deseje escalar para 64 ou 128 bits.
5. **Arquivo vec_adder_full.vhd.** Para compatibilidade com o simulador Digital, que não aceita múltiplos arquivos VHDL no componente externo, foi gerado um arquivo unificado concatenando cla4.vhd e vec_adder.vhd.

7 Análise de Recursos

O enunciado estabelece a quantidade de recursos gastos como critério de avaliação. A Tabela 5 compara as abordagens RCA e CLA para o bloco de 4 bits.

Tabela 5: Comparação de recursos: Ripple Carry vs Carry-Lookahead (4 bits)

Arquitetura	Portas lógicas	Níveis lógicos	Latência (gate delays)
Ripple Carry (RCA 4b)	~14	$2n = 8$	$8 \times t_g$
Carry-Lookahead (CLA4)	~22	2 (AND+OR)	$2 \times t_g$

O CLA usa aproximadamente 60% mais portas que o RCA equivalente de 4 bits, mas reduz a latência de $8 \times t_g$ para $2 \times t_g$ — uma melhora de $4\times$. Para o somador vetorial completo de 32 bits, a comparação é ainda mais favorável: o CLA mantém latência constante de $2 \times t_g$ independentemente de n , enquanto o RCA cresceria para $64 \times t_g$.

O projeto não adiciona recursos além do estritamente necessário: a lógica de máscara consiste em apenas 7 portas AND (os gates de carry) e 3 tipos de equações combinacionais distintas para **mask**, totalizando menos de 20 portas adicionais sobre os 8 blocos CLA4.

8 Conclusão

O projeto resultou em um somador/subtrator vetorial de 32 bits funcional, atendendo a todos os requisitos do enunciado. A arquitetura modular com 8 blocos CLA4 e controle combinacional de carry demonstrou ser eficaz e elegante: a mudança de comportamento entre os quatro modos vetoriais é obtida com apenas 7 portas AND e 3 equações lógicas distintas para a máscara.

A simulação com GHDL validou a correteza para todos os modos e operações, incluindo os casos críticos de isolamento de carry entre lanes adjacentes. A verificação no simulador Digital confirmou o comportamento esperado em modo interativo.

9 Referências

1. Patterson, D. A. & Hennessy, J. L. *Computer Organization and Design: RISC-V Edition*, 2^a ed., Elsevier, 2021.
2. Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual*, Volume 2 (Instruction Set Reference). <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
3. Neemann, H. *Digital — Logic Designer and Circuit Simulator*. <https://github.com/hneemann/Digital>
4. Pedroni, V. A. *Circuit Design and Simulation with VHDL*, 2^a ed., MIT Press, 2010.
5. Texas Instruments. *SN74S181 — 4-Bit Arithmetic Logic Unit*. <https://www.ti.com/lit/ds/symlink/sn74s181.pdf>
6. Texas Instruments. *SN74S182 — Look-Ahead Carry Generator*. <https://www.ti.com/lit/ds/symlink/sn74s182.pdf>